Interpreters and Macros Meta

Computer Science Mentors 61A

November 18 – November 22, 2025

Example Timeline

- Ice Breaker [3 min]

  If you want, start off with an icebreaker of your choice!

  Also, there is a link to a feedback form at the bottom of this worksheet! It would be amazing if you could highlight this/fill it out yourself so we can improve the worksheets in the future!
- Mini-lecture: Interpreters [10 min]
- Q1: Link [6 min]
- Q2: Scheme Eval/Apply [4 min]
- Mini-Lecture: Macros [10 min]
- Q3: meta-apply [5 min]
- Q4: combine-num [7 min]
- Q5: and-odds [10 min]

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28

1 Scheme Interpreters

1. Based on the Link class that is below, write the scheme expression represented by the Link class. Assume that this is the Link class in Python that we will be using:

```
class Link:
    '''A Scheme list is a Link in which rest is a Link or nil.'''
    empty = ()
    def __init__(self, first, rest=empty):
        self.first = first
        self.rest = rest

    # There are also __str__, __repr__, and map methods, omitted here.

nil = Link.empty
>>> Link('*', Link(5, Link(Link('-', Link(10, Link(2, nil))), nil)))
```

(* 5 (- 10 2))

```
>>> Link('or', Link(Link('>', Link(5, Link(2, nil))), Link(Link('/',
   Link(1, Link(2, nil))), nil)))
```

(or (> 5 2) (/ 1 2))

```
   >>> Link('+', Link(1, Link(Link('*', Link(Link('-', Link(5, Link(2,
       nil))), Link(Link('+', Link(3, Link(1, nil))), nil))), Link(5,
       nil))))
```

(+ 1 (* (- 5 2) (+ 3 1)) 5)

Teaching Tips

- Explain the Link class thoroughly and if you have time, explain what its connection is to interpreters and the scheme project.

Suggested Time: 6 min

2. Circle the number of calls to `scheme_eval` and `scheme_apply` for the code below.

```
(+ 1 2)
```

`scheme_eval`	1	3	4	6
`scheme_apply`	1	2	3	4

4 `scheme_eval`, 1 `scheme_apply`.

Suggested Time: 5 min

2 Macros

1. The built-in **apply** procedure in Scheme applies a procedure to a given list of arguments. For example, `(apply f '(1 2 3))` is equivalent to `(f 1 2 3)`. Write a macro procedure `meta-apply`, which is similar to **apply**, except that it works not only for procedures, but also for macros and special forms. That is, `(meta-apply operator (operand1 ... operandN))` should be equivalent to `(operator operand1 ... operandN)` for any operator and operands. See doctests for examples.

```
; Doctests
scm> (meta-apply + (1 2))
3
scm> (meta-apply or (#t (/ 1 0) #f))
#t
(define-macro (meta-apply operator operands)


)
```

```
(define-macro (meta-apply operator operands)
    (cons operator operands))
```

This is supposed to be a relatively gentle introduction to the process of writing macros. Nothing much to see here. Though there may be some students who want to just write `(operator operands)`. The issue with this, of course, is that it is treated as a call expression in the body of the macro, which causes an error. An alternate solution with quasiquote is also possible:

```
(define-macro (meta-apply operator operands)
    `(,operator ,operands))
```

Suggested Time: 5 min

2. Implement the macro function `combine-num`, which takes in an unquoted list of positive integers and returns a number whose digits are the items of the list in reverse order.

```
;Doctests
scm> (combine-num (1 2))
; 21
scm> (combine-num (2 5 3 5))
; 5352
scm> (combine-num (1))
; 1
scm> (+ (combine-num (1 2 3 4)) 5)
; 4326      # (4321 + 5)


(define-macro (combine-num lst)



)
```

```
(define-macro (combine-num lst)
  (if (null? lst) 0
    `(+ ,(car lst)(* 10 (combine-num ,(cdr lst))))
  )
)
```

Suggested Time: 7 min

3. It's Thanksgiving, and we need to ensure the odd-indexed dishes on the table pass the "taste test"! Write a macro, **and**-odds, which takes in a list of dishes, exprs, and evaluates to a true value only if all of the odd-indexed elements of exprs evaluate to true values. If any of the odd-indexed elements evaluate to false, **and**-odds should return false.

   Can you help us determine which dishes make the cut for the Thanksgiving feast?

```
; doctests
scm> (and-odds '((= 10 10)))
#t
scm> (and-odds '((= 1 2)))
#f
scm> (and-odds '(#f #t #t))
#f
scm> (and-odds '((< 5 3) (= 5 5)))
#f
scm> (and-odds '((> 3 2) (< 5 0) (= 5 5)))
#t
scm> (and-odds '((< 1 5) (< 5 2) (< 3 5) (< 5 3) (< 4 5)))
#t
scm> (define a (list 1 #f 3))
a
scm> (and-odds a)
3
(define-macro (and-odds exprs)
    `(if _____________________________________________________

         _____________________________________________________

         _____________________________________________________
    )
)
```

```
(define-macro (and-odds exprs)
    `(if (> (length ,exprs) 2)
         (and (car ,exprs) (and-odds (cdr (cdr ,exprs))))
         (eval (car ,exprs))
    )
)
```

Teaching Tips

- Quickly review boolean evaluation and boolean operations **and** and **or**.
- Begin with the intuition behind the recursive call. Have them consider an arbitrarily long list and guide them to the intuition behind the call on (cdr (cdr exprs)).
- If they add a base case for nil, have them consider an odd-length list.

Suggested Time: 10 min